

Project of Web Recommender Systems

Yicheng Feng

MS in IT and Cognition, KU

zhp525@alumni.ku.dk

1 Dataset Preparation

The "Musical Instruments" dataset is from Amazon product review datasets, which contains user reviews, item IDs, ratings, and timestamps. The full dataset contains around 213000 entries.

In this project, we used 14000 entries from it to save computational cost, and it is still large enough to implement our collaborative filtering, content based and hybrid recommender systems.

1.1 Data Split and Cleaning

The first step is to split data into training and test data, which are already provided in this project. The numbers of entries in training and test data are, respectively, 80% and 20% of the project dataset.

Both training and test data are cleaned by removing entries with missing user or item ID, removing users from test set but not in training set, and removing duplicates with same user ID and item ID by keeping only the last occurrence. After cleaning, the structure of dataset are as follows in Table 1, with 1087 duplicates removed from training set and 1178 duplicates removed from test set:

	Entry count	Item count	User count
Full set	11735	518	800
Train set	9913	509	800
Test set	1822	413	437

Table 1: Overview of cleaned dataset

1.2 Overview of user/item statistics

I plotted the frequency of ratings for each item and the frequency of ratings by each user from training set, which are shown in Figure 1.

The two plots both follow a long-tail distribution, indicating a significant property of the dataset that we need to consider in future experiments and evaluation. We can see from figures that a few popular items receive more than 50 ratings, while most

items are rated less than 20 times. Meanwhile, a small number of users rate many items, while most users rate 10 to 20 items. Hence we know that many users and items have insufficient data, likely causing cold-start problem and might only recommend most frequently rated items while omit items with high rating but few frequencies.

1.3 Top 5 highly rated items

Here I computed the frequencies of highly rated items in training set whose rating is higher than or equal to 3. Table 2 below is the details of top 5 highly rated items. Item IDs are given along with

Item ID	Rating Freq	Average
B07N2HQ1T7	28	5.0
B0742RB7JK	14	5.0
B07C9YCY5J	14	5.0
B000U0DU34	13	5.0
B0BT2W3TTM	13	5.0

Table 2: Top 5 highly rated items.

their counts of ratings and average rating. We can notice that their rating frequencies are low and all located in the 'long tail', thus more rating counts does not necessarily correlate with a higher average rating. This may because a more diverse group of users has a wider range of opinions, leading to greater variability in ratings. But fewer users could result in circumstantial result which might not be well served as a reference. Thus in real-life recommendation, more factors need to be considered, depending on whether we want a more reliable recommendation or risky but worthwhile one.

2 Collaborative Filtering Recommender System

2.1 Choice of CF Recommender System

I selected one neighborhood-based model, k-nearest neighbors (KNN), and one latent factor model, Singular Value Decomposition (SVD).

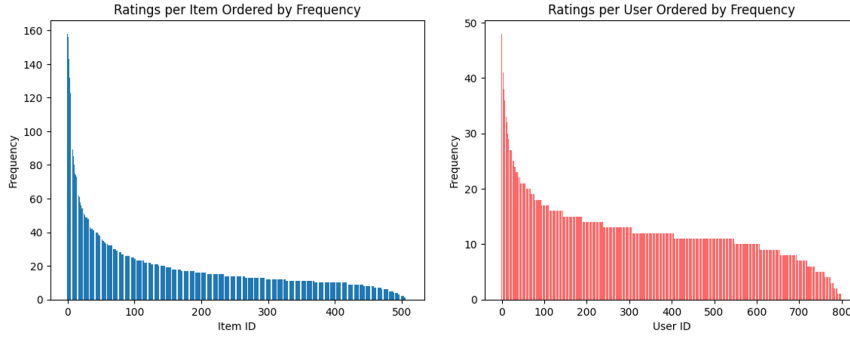


Figure 1: Item-rating frequency and User-rating frequency.

KNN pblackicts the value of a data point by finding the k closest data points in the training set and taking an average via regression. And SVD pblackicts data by decomposing a matrix into latent factors and using these factors to estimate.

2.2 5-Fold Cross-validation

As parameters of model significantly influence the performance, fine-tuning is necessary. I used 5-fold cross-validation to select the best hyperparameters for each model. Here cross-validation can ensure that chosen parameters generalize well to unseen data, preventing over-fitting, and 5-fold spitted the train dataset into five subsets and iteratively training on four while validating on the fifth.

Specifically, number of neighbors and similarity measures are chosen parameters for KNN model and number of latent factors and epochs for the SVD model. Below are the parameters and their tested value chosen based on feature of dataset.

For KNN:

- Number of neighbors: $\{1, 2, \dots, 10\}$.
- Similarity measure: $\{\text{cosine}, \text{pearson}, \text{msd}\}$.

For SVD:

- Number of factors: $\{5, 10, 20, 30, 50\}$.
- Number of epochs: $\{10, 20, 30, 50, 100\}$.

Here "cosine" refers to cosine similarity, which measures the cosine of the angle between two vectors, focusing on their direction; "pearson" refers to Pearson correlation coefficient, which measures the linear relationship between two variables; and "msd" to Mean squablack-difference, quantifying the average squablack difference between two vectors.

2.3 Evaluation Measure for fine-tuning

I chose Root Mean Square Error (RMSE) as evaluation measure for 5-fold cross-validation process. It is for evaluating the accuracy of pblackictive models. Equation 1 below shows the calculation of RMSE.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (1)$$

where y_i is the actual rating, $\hat{y}_i$ is the pblackicted rating, and n is the total number of ratings.

The reason for choosing RMSE is that it penalizes larger errors, making it effective for evaluating rating pblackictions in the project. And 5-fold cross-validation is to blackuce bias and provide more generalized estimate. Thus averaging RMSE across folds can help find the best parameters to prevent over-fitting issue and improves model's reliability.

There are other evaluating measures such as nDCG which is a rank based measure. However rank based measures are more appropriate for recommender systems producing ordeblack lists of items, where the focus is on the ranking quality rather than the pblackicted ratings. In this section our goal is to pblackict accurate ratings for user-item interactions thus we chose RMSE.

2.4 Model Fine-tuning

With 5-fold cross-validation method and Root Mean Square Error as evaluation measure, we performed them separately on both KNN and SVD model. The optimal hyperparameters are selected based on the average RMSE scores, the lower, the better. Optimal hyperparameters and measure scores are listed below in Table 3.

KNN Parameter	Optimal	Min.Avg.RMSE
#neighbors	6	0.9029
Similarity method	'pearson'	
SVDParameter	Optimal	Min.Avg.RMSE
#factors	30	0.8423
#epochs	20	

Table 3: KNN,SVD Optimal Parameters and Score

2.5 Training and Pblackiction

Using calculated optimal parameters, the two models were trained on training data and pblackicted unobserved ratings respectively. Table 4 displays the number of pblackictions and the average pblackicted ratings for each model. The two model shows

Model	Pblackiction Number	Mean Pblackicted rating
KNN	217724	4.504
SVD	217724	4.498

Table 4: Pblackiction result for two models.

a very similar average pblackiction ratings over unobserved ratings. It indicates that they capture similar user-item-rating patterns among the rated entries data. However it can just be served as an overview of pblackictions and need to be performed for specific users and items for further recommendations.

Hence I also used the two models to rank non-rated items for each user as a rough recommendation. For display, I selected a user whose ID is 'AE23LDQTB7L76AP6E6WPBFVYL5DA', and top 10 recommended item for him as an example, to show the difference in recommendation between the 2 models. Table 4 is the result.

Rank	KNN (Item, Rating)	SVD (Item, Rating)
1	B000NGVQKO, 4.8	B07DK59QNR, 5.0
2	B005PGGU9O, 4.8	B07N2HQ1T7, 5.0
3	B00CPLODUU, 4.8	B00721BB44, 5.0
4	B00RX5HQS4, 4.8	B0BT9R8MMV, 4.97
5	B00VSYN25M, 4.8	B0BQ4HSMC9, 4.97
6	B0150YC54E, 4.8	B09857JRP2, 4.96
7	B015QK3GUO, 4.8	B00WT7KPCK, 4.96
8	B06XB3FQKB, 4.8	B0719KM5Y8, 4.96
9	B0719KM5Y8, 4.8	B00RVE9X06, 4.95
10	B07BHJH2Y8, 4.8	B07YK57N2M, 4.95

Table 5: Top-10 recommendations for one User

From Table 5 we can see that, the two models gave the same user completely different top 10 recommended items with different ratings. It is due to the different recommendation mechanisms. KNN model in this project is based on the similarity between users which is tend to recommend items

liked by similar users, while SVD model that calculates latent factors hard to be observed between users and items has a different recommendation.

3 Evaluation of Recommender Systems

3.1 Error of Pblackicted Likelihood of Ratings

First, I calculated the error of pblackicted likelihood of rating for items on each recommend model using the metric of RMSE. The result is shown in Table 6.

Model	KNN	SVD
RMSE Score	0.5344	0.3438

Table 6: RMSE score of models on test data split.

The RMSE score for latent factor model SVD is prominently lower than the neighbor-based model KNN, which showcases that SVD model has a better pblackictive accuracy on our test data. Although RMSE can show the pblackictive ability of a model, it is limited in measuring how well the model ranks recommended items and it is sensitive to large errors. Further more it might fail in sparse data.

3.2 Evaluation of Recommendations

Thus we need to introduce more metrics to have a more comprehensive evaluation for the models to find the best recommendations. I used five more metrics which are average precision, mean average precision, mean reciprocal rank, hit rate and coverage using $ratings \geq 3$ on test set to further evaluate the model. Computed scores of the five metrics for two models and the baseline model TopPop are listed in Table 7.

From the result we can see that the baseline model TopPop performs the worst, which recommends the same items for all users. And for average precision, mAP and mrr, both KNN and SVD model got a small and similar scores which show that neither of them performs really well. For hit rate, KNN model got 0.105 while SVD model got 0.112, indicating that SVD model gives more precise recommendation. But there's KNN model has a significantly higher coverage than SVD, showing that KNN model recommends more diverse items than SVD model does, and SVD model only focuses on a small scale of items.

Combining RMSE scores, the two models doesn't give perfect good recommendation but better than non-diversed TopPop, while SVD model

Model	AP@10	mAP@10	mrr@10	hit rate@10	coverage@10
KNN	0.013	0.002	0.019	0.105	0.807
SVD	0.012	0.002	0.020	0.112	0.255
TopPop	0.005	0.000	0.005	0.047	0.019

Table 7: 5 metric scores of KNN,SVD and baseline TopPop on test data split.

has a slightly higher precision and KNN model has more diverse recommendation that might recommend items that are not popular but with good quality. But considering that the hit rates are similarly low, KNN might does a better job in recommendation. That might be because KNN model recommends items based on similarity between users so that unpopular items liked by one user could be recommended to other similar users. While SVD model that is latent-factor driven, focuses more on user's preference and the hidden user-item connection, giving a more precise recommendation that considers many factors but lower diversity. Although TopPop is more efficient to generate recommendations, it is the least plausible in real world problems.

3.3 Limitation of Evaluation measures

Although RMSE and five metrics tells certain differences between models, most of them focus on recommendation precision but not diversity. With only coverage to measure diversity, it is hard to have a solid conclusion. Meanwhile with many metrics implying precision, it is hard to trade-off and combine all of them.

4 Text Representation

I chose to use subset of metadata which only contains items from training and test set. Because we want to compare text representation methods and using a subset is more efficient, also preventing overfitting if any. Total item number in metadata file is 23984 and item number here is the unique items from training and test set which is 518.

4.1 Metadata Preprocessing

After filtering items in metadata file, we need to preprocess text content from 'description' column. At first I extracted the descriptions and removed empty description entry(97 entries in total), then I performed transformation to lowercase, punctuation removal, tokenization and stopwords removal in turn. The reason for the preprocessing steps is to ensure case insensitivity, blackuce unnecessary noise, break text into tokens for further

processing and blackuce vocabulary sizes for efficiency. I did not implemented stemming because it destroys the semantic meaning of the word and does not work well with word embedding methods. The item number and vocabulary size before and after preprocessing is listed below in Table 8.

	Original	Preprocessed
Item count	518	421
Vocabulary size	5996	5802

Table 8: Item and voc size before,after processing

4.2 Item Representation

In this project I experimented 3 type of item representation on pre processed descriptions, which are TF-IDF, word-2vec and pretrained BERT.

TF-IDF is based on statistics, composed of Term Frequency and Inverse Document Frequency, it can capture key information but not semantic features. Word-2vec is a word embedding technique based on neural network, mapping words to low-dimensional vector space and capturing semantic features. And pretrained BERT is a language model based on transformer, capturing contextual information bidirectionally and word vectors from the last layer of BERT is used to represent items.

I did TF-IDF and word-2vec representation on preprocessed data and get a TF-IDF matrix with shape (421, 5743) and an item-embedding dictionary with 419 item-embedding pairs, respectively. I also did pretrained BERT representation on data which only filteblack out empty entries, as pretrained BERT has its own tokenizer, and the final embedding of an item is the average embedding of all the processed words in the description. I got the average embedding with shape of (421, 768).

4.3 Similarity Between Items

Now I explblack the similarity between items within vector space for the three representation methods. Here I computed the cosine similarity matrix.

For detailed desplay, I chose three items to show in Table 9 below. From the overview of three matrix and three items we can find that TF-IDF cosine

TF-IDF	item1	item2	item3
item1	1.000000	0.000000	0.033442
item2	0.000000	1.000000	0.000000
item3	0.033442	0.000000	1.000000
Wore-2vec	item1	item2	item3
item1	1.000000	0.331785	0.572373
item2	0.331785	1.000000	0.419767
item3	0.572373	0.419767	1.000000
BERT	item1	item2	item3
item1	1.000000	0.544365	0.711087
item2	0.544365	1.000000	0.573865
item3	0.711087	0.573865	1.000000

Table 9: Cosine similarity of 3 items: B0B8M5FJ9W, B0002E2G5Q, B07HGRFG5J

similarity is very sparse and most values are 0; while matrix of word-2vec and BERT embedding are more dense and with less zero value. Also values in BERT matrix are higher than the other two. That indicates that TF-IDF representation has limited semantic understanding as it bases on frequency but not context. Word-2vec and BERT both can capture semantic relationships between items while BERT consider bidirectional information and thus captures more information, outperforming other representation methods.

5 Content-Based Recommender System

I used the same subset as in Section 4 with items included in training and test sets, so that it is more proper when comparing all recommender systems.

5.1 Features Preprocessing

In this content-based recommender system I chose both 'description' and 'title' to represent items into vector space. For 'description' and 'title' column, I applied these pre processing technique: **lowercase, tokenization, removal of punctuation and stop-words, and stemming**. As I will transform them into TF-IDF representation thus I applied stemming in this section in order to blackuce vocabulary size. Item count, vocabulary size of descriptions and titles before and after processing are in Table 10.

	Original	Preprocessed
Item count	518	421
Description vocabulary	5996	4470
Title vocabulary	1630	1476

Table 10: Item and voc size before,after processing

5.2 Item and User Representation

After pre processing, I first represented the all the items from training and test set into TF-IDF vector

space. I respectively transformed 'description' and 'title' column into TF-IDF representation, and then combine them using concatenation. The shape of TF-IDF matrix of description is (421, 4420) and of title is (421, 1455), the shape of their combination is (421, 5875), which is the shape of final item representation. Then I represented each user in the same vector space. My method is to compute weighted average of vectors of items from training set that are rated by each user as shown in Equation 2.

$$\text{user_vector} = \frac{\sum_{i=1}^n (\text{item_vector}_i \times \text{weight}_i)}{\sum_{i=1}^n \text{weight}_i} \quad (2)$$

Here the weight of item is its rating by each user.

5.3 Content-Based Pblackiction

With item and user representations, now I calculated user-item pblackiction for each item from both training and test set that are not rated, based on the similarity between users and items, and recommended top 10 items for users with the results.

The method I used for calculating similarity between users and items is **cosine similarity** which measures the cosine of the angle between two vectors. The reason is that the item representation which based on TF-IDF is high dimensional and sparse and cosine similarity is robust to sparsity as it ignores the magnitude. It is not proper to measure distance that might mislead recommendations.

Thus I first computed the cosine similarity matrix for items and users, then I used the result to pblackict user-item ratings for unrated items. User-item pblackicted rating is computed using the Equation 3.

$$R_{\text{pblackict}} = \bar{r} + \cos \times (r_{\max} - r_{\min}), \quad (3)$$

where $\bar{r}$ is the mean rating of items rated by each user, $\cos$ is the cosine similarity between user and item vectors, $r_{\max} = 5$, the maximum possible rating and $r_{\min} = 1$, the minimum possible rating.

5.4 Evaluation of Recommendation

With pblackicted ratings and test data, I evaluated this content-based recommender system by recommending the top 10 highly rated items from pblackiction for each user and calculating evaluation metrics based on recommendations and items where $\text{ratings} \geq 3$ from test data. Evaluation metrics used is the same to that in Chapter 3.2. Results are shown in Table 11 along with models in Chapter 2.

From Table 11 we can see that the content-based recommender system has highest average precision, mAP, mrr and hit rate with cutoff 10, while only its coverage is lower than collaborative filtering recommender system based on KNN. That indicates that content-based system's recommendations are precise, and performs better across multiple users and ranks the first relevant item higher on average, but recommends fewer unique items compared to the collaborative filtering system based on KNN, but more than other models. That is because Content-based system relies on item features and user preferences, leading to more precise recommendations but collaborative filtering systems only focus on interaction statistics but not features; lower coverage is due to that if the feature space is narrow or lacks diversity, the system may repeatedly recommend similar items. Generally content-based recommender system based on TF-IDF representation performs the best among implemented models.

6 Hybrid Recommender System

6.1 Introduction of systems

I implemented 3 hybrid recommender systems below and evaluated them in the same way as in previous chapters. In all the 3 hybrid systems, I combined collaborative filtering system based on SVD(CF-SVD for short) and content-based system based on TF-IDF(Content-TF for short) representation. The reason to choose CF-SVD is that it is more suitable for sparse data than CF-KNN, thus cooperates better with content-based systems.

Parallel combination recommender system combines recommendations from multiple models simultaneously using an aggregating method. I combined the recommendations from CF-SVD

and Content-TF by computing average pblackicted rating of items from the two models and re-ranking them to get new top 10 recommended items.

Switching recommender system switches between different models based on conditions or contexts. I used user interaction frequency as the condition where $threshold = 10$: if a user rated items more or equal than 10 times, then use the recommendations from CF-SVD; if less than 10 times then apply recommendation from Content-TF. My reason is that for users with fewer interactions, CF-SVD may perform poorly due to insufficient data and switching to Content-TF ensures that these users still receive good recommendations based on item features. And for users with sufficient interactions CF-SVD can provide high-quality recommendations. Thus it can solve cold-start problem.

Pipelining recommender system uses one model to generate an initial set of recommendations, then refine/filter them by another model. Here I first used Content-TF to generate recommendations, then did filtering with $rating_{pblackict} \geq 3$, then I used CF-SVD to generate new recommendations from the filtered recommendations for each user. That is to ensure that semantically related items won't be filtered out in the first place.

6.2 Evaluation of Recommendation

Calculated evaluation metrics for the three hybrid recommender systems and all the models in previous chapters are shown in Table 12.

From the the table of all the recommender systems implemented in this project it is more intuitive to compare and analyze them. Among hybrid recommender systems, regarding precision of recommending, parallel combination strategy performs

Model	AP@10	mAP@10	mrr@10	hit rate@10	coverage@10
Content-based	0.013	0.002	0.021	0.132	0.633
KNN CF	0.013	0.002	0.019	0.105	0.807
SVD CF	0.012	0.002	0.020	0.112	0.255
TopPop	0.005	0.000	0.005	0.047	0.019

Table 11: 5 metric scores of content based and CF models, baseline TopPop on test data split.

Model	AP@10	mAP@10	mrr@10	hit rate@10	coverage@10
Parallel	0.015	0.002	0.021	0.134	0.515
Switching	0.013	0.002	0.021	0.132	0.633
Piplining	0.010	0.002	0.020	0.096	0.264
Content-based	0.013	0.002	0.021	0.132	0.633
KNN CF	0.013	0.002	0.019	0.105	0.807
SVD CF	0.012	0.002	0.020	0.112	0.255
TopPop	0.005	0.000	0.005	0.047	0.019

Table 12: 5 metric scores of content based and CF models, baseline TopPop on test data split.

the best with highest average precision, mAP, mrr and hit rate; second best is switching strategy and pipelining strategy has the lowest precision. The result might be due to that parallel combination strategy leverages the strengths of both models, leading to more accurate and robust predictions while switching strategy can not fully combine multiple models simultaneously due to the bias of conditions. Meanwhile the pipelining strategy performs worst which might be due to its limitation to correct mistakes made on previous recommendation filtering. While regarding diversity, switching strategy performs best and also pipelining strategy performs worst. That might be due to switching strategy switches between models that focus on different aspects of recommendations while pipelining strategy narrows the range through filtering.

Among all the systems, **parallel combination system still has the highest precision while the collaborative filtering system based on KNN has the highest diversity in recommendation**, due to KNN's reliance on user interactions and neighborhood exploration. Also switching system has very similar scores with content-based system, that might be due to that most users have little interactions thus Content-TF influenced the result mainly. Generally, hybrid recommender system performs better as it combines strengths of multiple models, if a good strategy like parallel combination chosen. Moreover, Parallel Combination is the best if precision is the primary goal while its diversity can be improved by refining its hybrid methods.

7 Long-tail Effect Analysis

To visualize longtail effect, I plotted figures for hit rate of top and last 20% users based on their rating frequency and for coverage of top and last 20% items based on rating frequency, shown in Figure 2.

From the two figures we can see, for long-tail effect for users, CF systems exhibit significantly higher hit rates for less active users compared to active users, showing effectiveness in catering to long-tail users but fails to active users. While content-based and hybrid recommender systems show balanced performance across both user groups, suggesting ability in handling multiple user types. TopPop performs poorly for both groups. For long-tail effect for items, KNN based CF got the highest coverage for both the popular items and tail niche items while SVD based CF per-

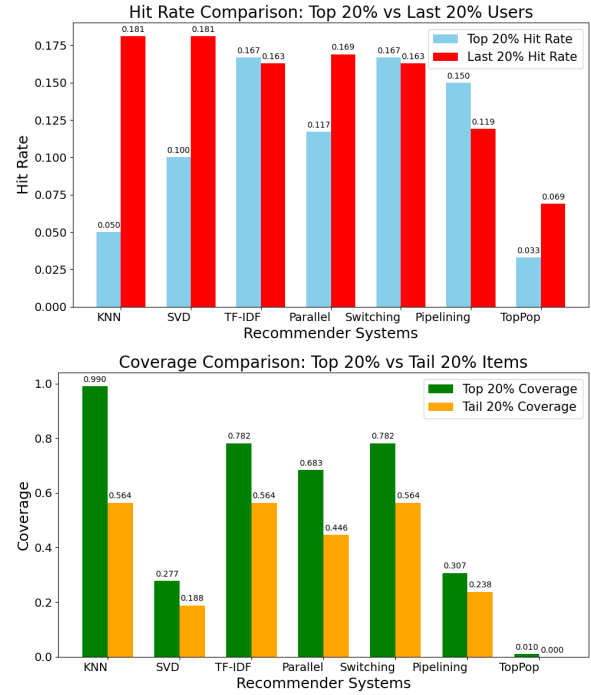


Figure 2: Plots for Longtail Analysis.

forms bad. content-based and hybrid recommender systems show relatively strong coverage for both groups with a smaller gap, except for the one using pipelining strategy. TopPop fails on both groups of items. Overall, **content-based recommender system based on TF-IDF and switching system do the best for blackucing long-tail effect**, balancing performance across different users, popular and niche items. **KNN based CF system does well for blackucing item long-tail effect but still needs to be improved for user long-tail effect.**

8 Neighborhood Based CF Error Analysis

To analyze error of neighborhood based CF, I chose user 'AEMS FZ4NLED647BP3ZCQEY3FNGKA' with $RR = 1$ and user 'AE2W7OBJ45J-4J52CGH7OFWHQZJQA' with $RR = 0.101$ from training set as reference users. With implemented KNN based CF, I retrieved the 10 nearest neighbours of them with their rating history. From rating history of neighbors and calculation of overlaps of ratings between reference users and their neighbors, I got 28.5% of overlap between the reference user with higher RR and its neighbors, while 71.4% of overlap between the user with lower RR and neighbors. The model performs well for high RR users even the overlap with their neighbors is low, and it performs worse for lower RR users even the overlap with their neighbors is high. It shows that higher

498 overlap does not guarantee good recommendations
499 and the quality of the overlap might be more im-
500 portant than the quantity. Also it indicates that
501 there is probably **a bias in model** that benefit the
502 high RR user more, and the model fails for users
503 with **sparse or noisy data**, which are needed to be
504 considered for error balancing.

505 **9 Conclusion**

506 From the project, I have a deeper insight into rec-
507 ommender systems. Moreover more aspects need
508 consideration in a real-life recommendation.